



Assignment: The Auld Argument Shoppe

Course: PA1489 Basic Software Engineering
Date: 2026-06-15
Author: Mikael Svahnberg
E-mail: Mikael.Svahnberg@bth.se



Introducing The Auld Argument Shoppe



The Auld Argument Shoppe is a microservice application used to exercise building and deploying microservices with e.g. Podman or Docker, and to orchestrate a local deployment of many collaborating services locally using Docker Compose.

Using the well-recognised example of a webshop, the application has been over-engineered to expose as many aspects of deployment as possible. At the same time, there are many things missing that are needed to make it a complete webshop. I leave it as a voluntary exercise (that does not contribute to the final grade) to identify these missing features and to extend the application accordingly.

Some intentional design choices are:

- All microservices listen on port 5000. This highlights that each container should really be seen as a separate mini-computer.
- Product data are located in a single, shared folder. This exercises the “volumes” spec during orchestration.
- Databases do not have persistent volumes to simplify restarts.
- In order to get as many microservices as possible, they have in fact ended up as too small to be meaningfully maintained. In fact, they are better named nanoservices and can be considered an antipattern.
- To further drive home the point that “there is no way in \$NETHERWORLD that I am going to install all of this on my local computer; there must be a better way!”, the microservices are written in a wide selection of programming languages (more on this later).

More info

- <https://codeberg.org/mickesv/ArgumentShop.git>
- <https://github.com/bth-diit-teaching/PA1489-Basic-SE.git>

About this Document

This is part of a course “Basic Software Engineering” offered at Blekinge Institute of Technology. This is a workbook for one of the assignments in the course. In this assignment we do not cover automated testing or using a debugger; this is covered elsewhere in the course.

The format may seem whimsical (Sorry, I got carried away), but the tasks are real and necessary in order to understand how the application is connected and for you to complete the missing parts so that everything actually runs. In case you want more motivation, this also contribute to your grade in the course.

You will complete some pages by hand. Please print these and show them to a TA during one of the lab sessions in order to get them marked.

Best of luck!

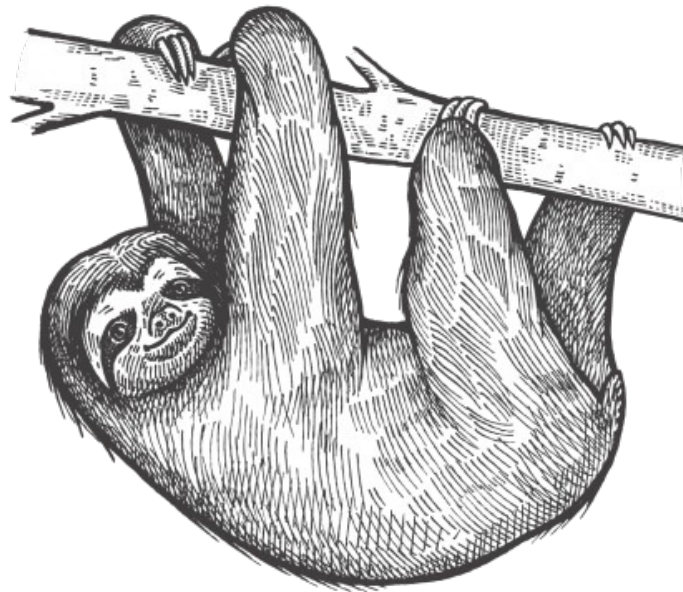
/Mikael

Mikael.Svahnberg@bth.se



Getting Started

1. Clone the Repository: <https://codeberg.org/mickesv/ArgumentShop.git>
2. Familiarise yourself with the application and its code.
3. Update your Engineering Diary:
 1. What are your initial reactions to this project?
 2. Is there anything you can see that you need to learn more about?
 3. What can you ignore for now?
4. Look in particular at the code for the “Webshop” container, and update your engineering diary:
 1. Do you understand how it works?
 2. Can you follow what happens when a call arrives via any of the routes? How are parameters extracted from the call? Which functions are called? Where do you find them? Do you understand how a web page is constructed and returned?



An Engineering Diary

As you work with new tools, or implement new code, an engineering diary is a vital help to be able to re-trace previous decisions, to note down quick ideas, or items that require further reading. In real life this engineering diary is your personal tool and life-saver, but in this course we also use it as part of the examination in order to bootstrap your use of a diary as a vital engineering tool.

- Try to get into the habit of reflecting regularly upon what you have done, what you have learnt, and what you must find out.
- It is better to write a little every day rather than a lot once a week.
- Even an entry such as "2025-06-22: Finished implementing the Frobnicator class; all went well and now I want to go out in the sun and play!" is valuable.

A Potluck Application

The Argument Shoppe is based on a number of collaborating microservices. Because of Bad Management™, every developer has used their own favourite language.



Webshop

I write in Python using Flask, because that is what I have been taught.

ProductCatalog

Go is the future! I also prefer a real database, so I use PostgreSQL.



ShoppingcartKeeper

I write in NodeJS because then I can use the same Javascript as in the web browser. I use MongoDB because I can't stand classic SQL Databases!

TaxCalculator

Rust is a low-level memory safe language. I use Rocket to create the REST API.



PackingSystem

If C++ was good enough for my parents, it is good enough for me.

DiscountCalculator

I also use Rust, but I like to build things from scratch!



CheckoutManager

Clojure is a dialect of lisp and an elegant tool for a more civilized age.

CustomerManager

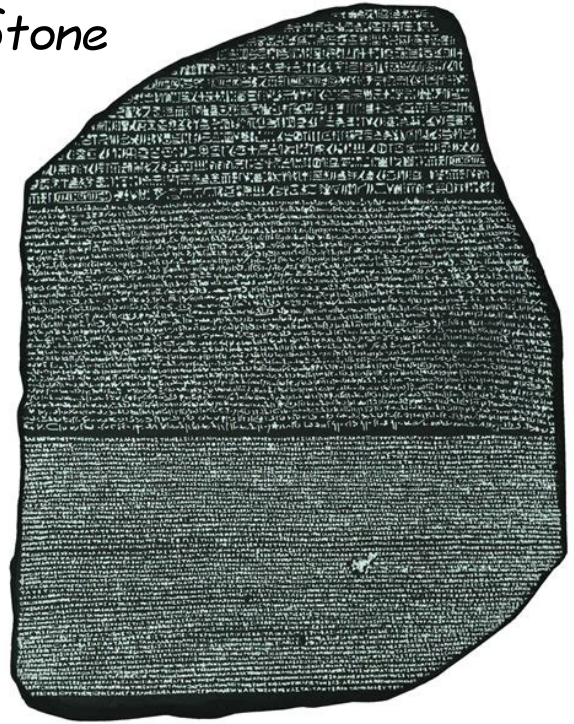
Sometimes when all you want is access to an underlying database there is no need to write it yourself. PostgREST solved my needs nicely.



Databases and Admin Interfaces

Ruff! I agree. With a PostgreSQL database as well as a MongoDB database, we need two database admin interfaces; adminer and mongo-express. But only during development!

The Rosetta Stone



The Rosetta Stone was a vital key for finally deciphering the ancient Egyptian hieroglyphic writing. The fragment is slightly taller than 1m and contains the same inscription written in two languages (Egyptian and Greek) and three writing systems (hieroglyphics, demotic script, and the greek alphabet).

Although I sometimes feel almost as ancient as the old Egyptians, I make no claim to be as significant as the Rosetta Stone. However, The Argument Shoppe bear some similarities to the Rosetta Stone since most of the microservices solve very similar tasks but in several different languages:

- They read files
- They connect to databases and query them
- They listen on network ports for HTTP calls
- They decode these calls, fetch information from files and databases, and construct JSON responses.

You are going to encounter these tasks and technologies again and again during your education, and you are going to encounter (or want to encounter) many of the programming languages as well. When you do, they may not seem quite as alien to you, and you may even want to revisit this assignment just to have one more example available to you to learn from. I have tried to keep the code as simple and bare-minimum as possible, so as not to cloud the underlying principles.

Tasks

1. Update your Engineering Diary:

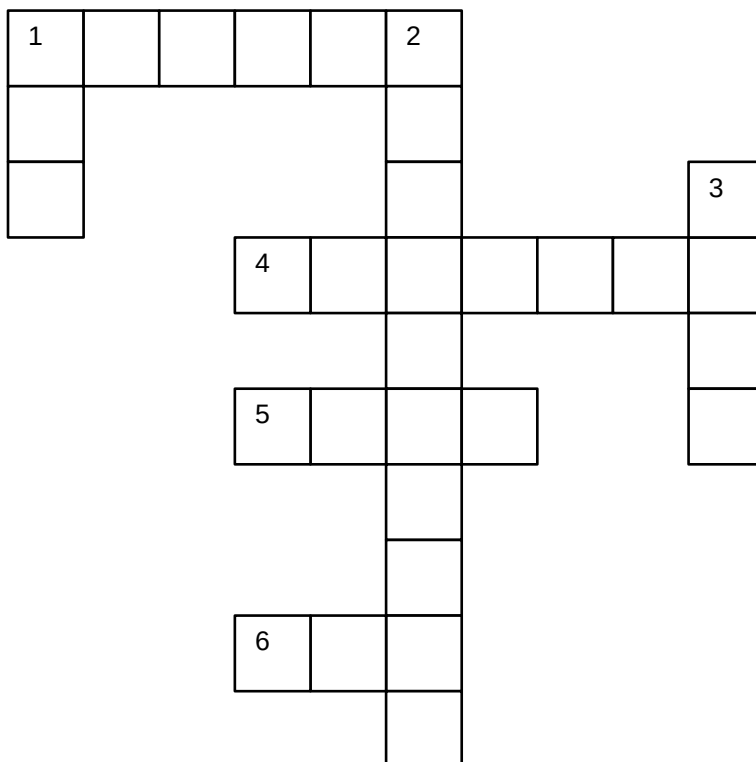
1. Add a brief summary of the following

- C++
- Node.js / JavaScript
- Rust
- Go
- Clojure
- SQL
- HTML
- HTTP



2. Are there any of these that you know of since before?
3. Are there any of these that you already know you want to learn more about?

Dockerfile Crossword



Across

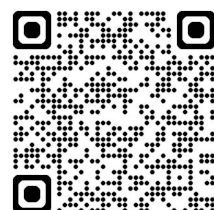
- [1] Open the following network port
- [4] Change active directory inside the image
- [5] Include a file from the host to into the image
- [6] Execute the following

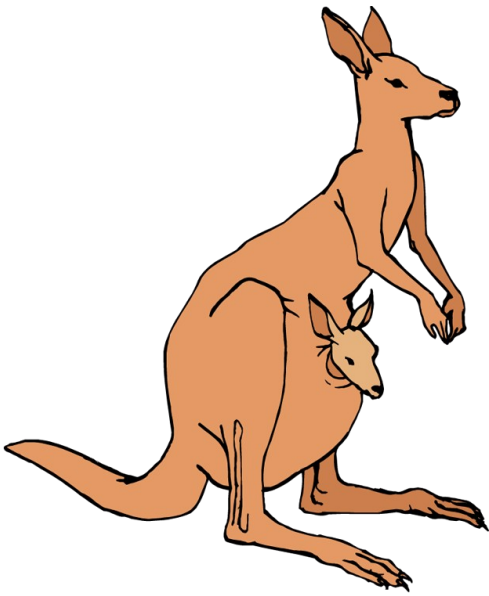
Down

- [1] Set an environment variable
- [2] The command that launches a container
- [3] Build the image with the following as base

Help Barry Hibernate

Barry cannot go to sleep unless he completes the crossword. Please help him!





Help Mother Roo Fix the Dockerfiles

Baby Roopert escaped and accidentally messed up two Dockerfiles. Help Mother Roo put them back together again!

Containers/PackingSystem/Dockerfile

```

__ gcc:latest

__ apt-get update
RUN apt-get -y install libpqxx-dev libcpp-httpplib-dev nlohmann-json3-dev

ENV PGPORT="5432"

EXPOSE __
__ /app
COPY . .
__ make init all
__ ["out/app"]
  
```

Containers/Webshop/Dockerfile

```

FROM python:3.8-slim

WORKDIR /app

COPY requirements.txt requirements.txt
RUN pip3 install -r requirements.txt

COPY . .
EXPOSE __
__ PYTHONUNBUFFERED="yes"

__ ["python3", "__"]
  
```

Tasks

1. Complete the Dockerfiles in the project
2. Make at least one commit for each Dockerfile
3. Build the Images
4. Create and run containers from the images

Show the TA

- The commits and the Dockerfiles
- Your command for building the images
- Your command for running the containers

Update your Engineering Diary

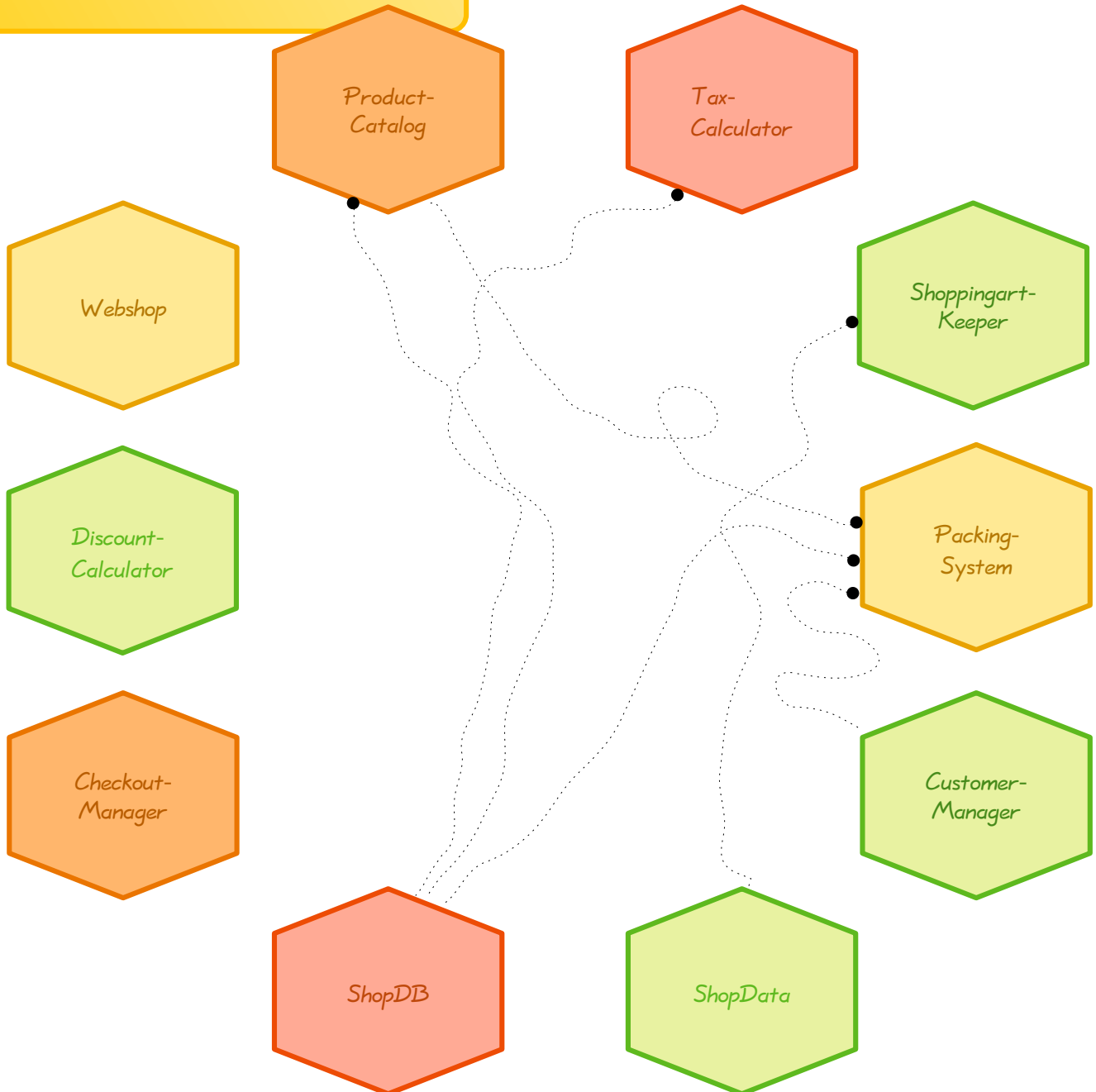
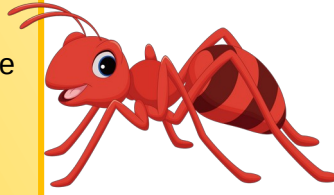
- Which Docker commands are you using?
- What works well? What is difficult?
- What do you need or want to learn more about?



Microservice Dependencies

Make a Dependency Map

Briant knows that all the dependencies are specified in the file *compose.yaml* . Help him complete a map of the system by drawing a line for each dependency.



Did you Know?

There is a difference between a dependency and a dependency. Sometimes, all you need is for another service to exist, and other times you need to be sure that the application within has properly started and is ready to receive calls. Using Docker Compose, you can use a short syntax where the services are created and removed in the specified order, or you can use a long syntax where you specify a *condition*, i.e. *service_started*, *service_healthy*, or *service_completed_successfully*. If you use *service_healthy*, that service must also implement a *healthcheck* .



Help Harriet fix compose.yaml

Harriet liked the compose file so much that she hid parts of it and now she can't remember where. Luckily, she vaguely remembers the contents. Help her rewrite the missing parts.

productcatalog:

image:

build:

depends_on:

volumes:

- I think all of the image names were the same as the service names.
- This one depended on *shopdb* being healthy.
- *./Products/Products* was mounted to */products*

Tasks

1. Edit *compose.yaml* and add the missing specifications.
2. Commit your changes.
3. Use *docker compose build* to build all the images.



Be forewarned! it will take some time to build all the images the first time.

shoppingcartkeeper:

image:

build:

depends_on:

Oh, I remember this one was special. It did not need any volumes and only depended on *shopdata* being started.

packingsystem:

image:

build:

depends_on:

environment:

Right. Here we go.

- *shopdb* must be healthy, but *productcatalog* and *customermanager* need only be started.

Four environment variables were set:

1. PGHOST to shopdb
2. PGDATABASE to productdb
3. PGUSER to postgres
4. PGPASSWORD to hunter2

taxcalculator:

image:

build:

depends_on:

volumes:

- This service depended on *shopdb* being healthy.
- *./Products/Data* was mounted to */products*



Can I just say that using the default user-name like this is a security threat, even though you never expose the database!

discountcalculator:

image:

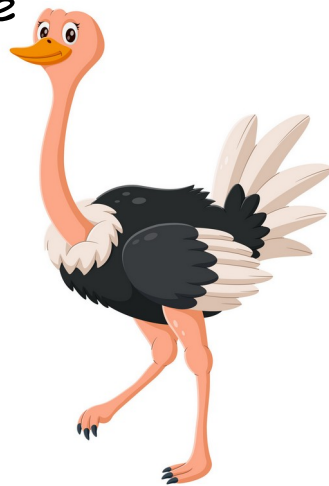
build:

volumes:

./Products/Data was mounted to */products*



Run The Auld Argument Shoppe



You should finally be at a point where it is possible to run the application and test it from a web browser. To recap:

1. You should have a complete *Dockerfile* for all containers.
2. You should have complete *compose.yaml* file where all services have image names and build instructions, and all necessary dependencies, volumes, and environment variables are specified.
3. You have used *docker build* to test the creation of all images.
4. You have used *docker compose build* to test the compose.yaml file.

Now, run it all using the command: *docker compose up*

... And wait.

... And wait.

Eventually, you should see the text “*webshop-1 | Listening on port 5000...*” in the console. This means that you can now open a web browser to

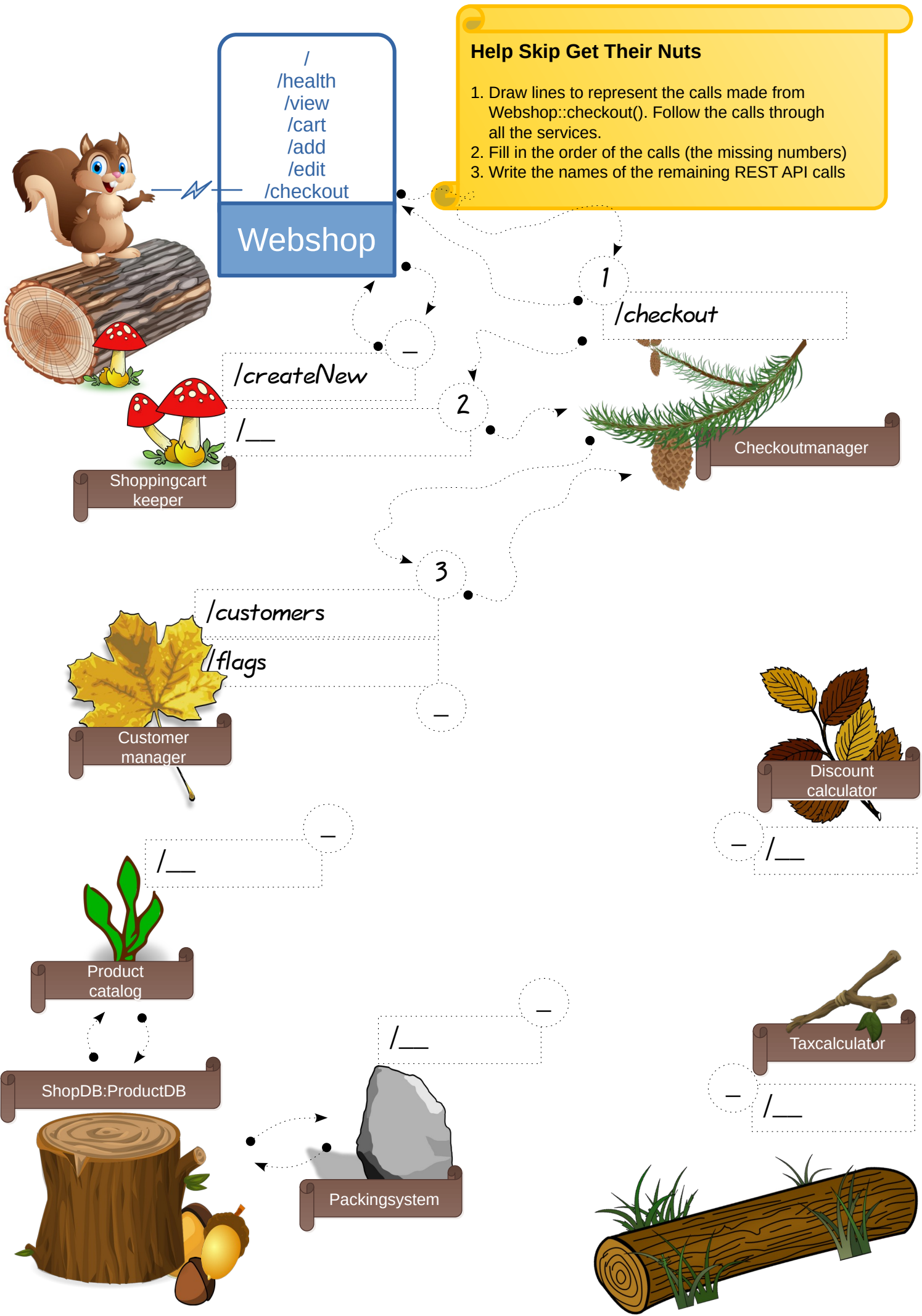
- <http://localhost:8080/>
- And: <http://localhost:8080/health>

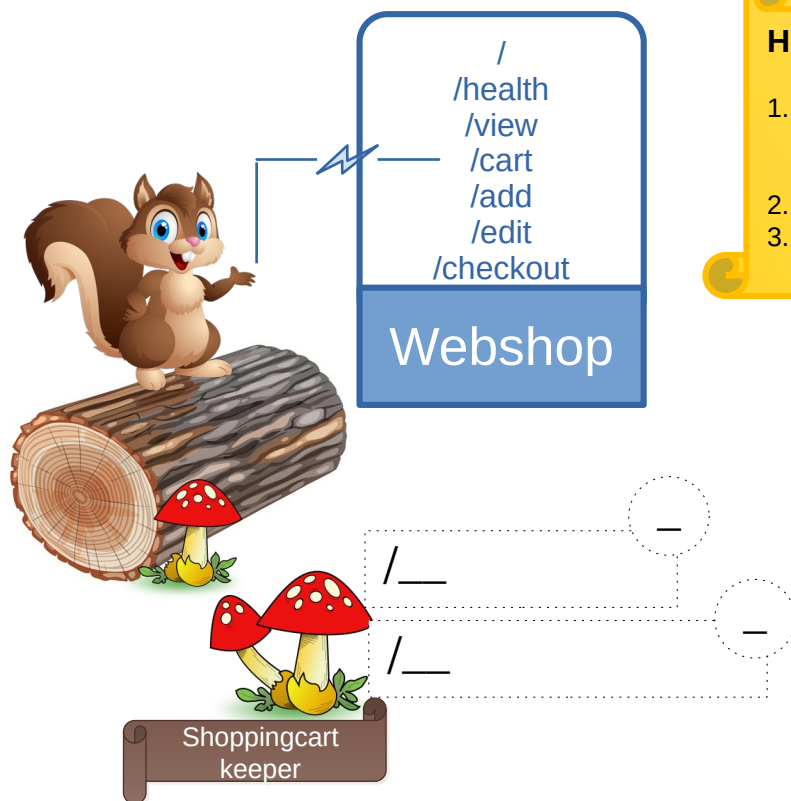
and test the web shop. In case you get timeout messages, try running the *up* command again. If this still fails, you may need to extend the interval to e.g. 30s in the HEALTHCHECK for the *tardy* container (and rebuild the image), before re-trying. The HEALTHCHECK is most likely specified in the service’s *Dockerfile*.

Things to Think About

What happens when you click on an “Add to Cart” button? What does the web browser send to the Webshop server? How does it know what to send? What do you think it gets back from the server? How did it get the updated number of items in the shopping cart?

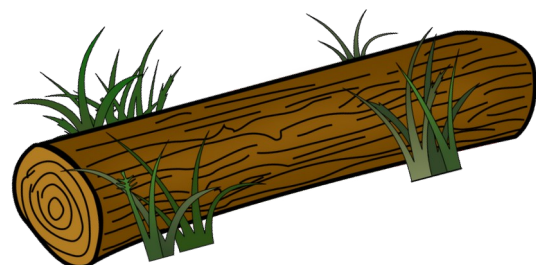
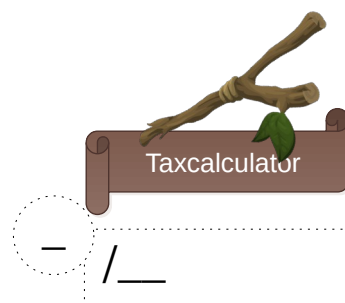
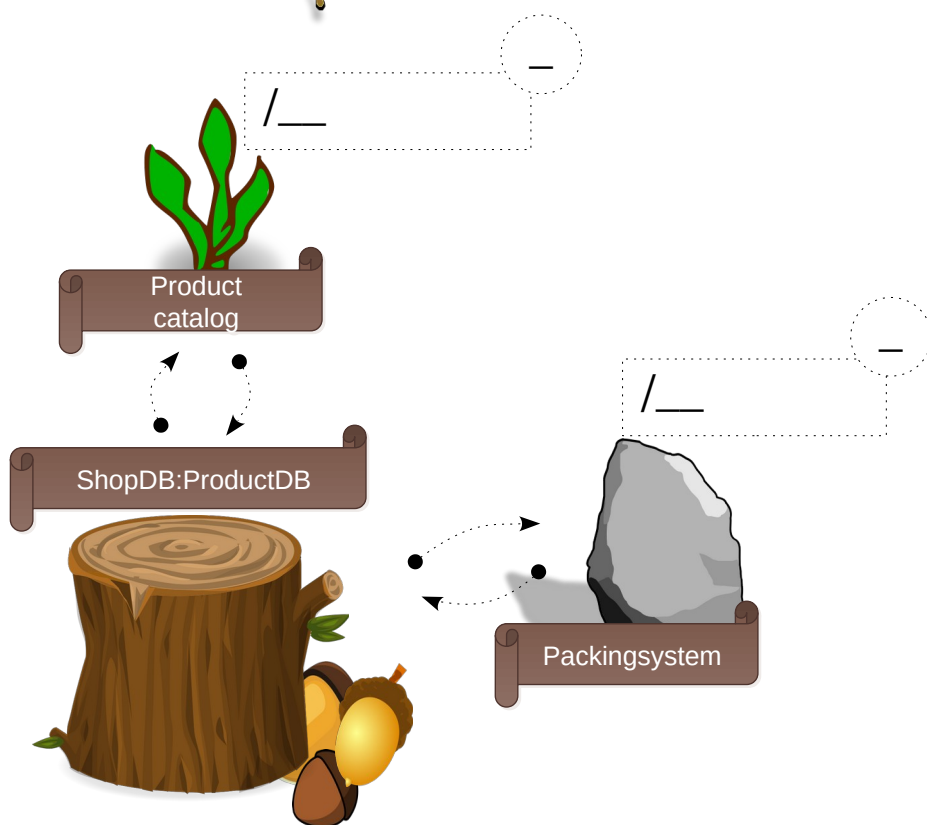
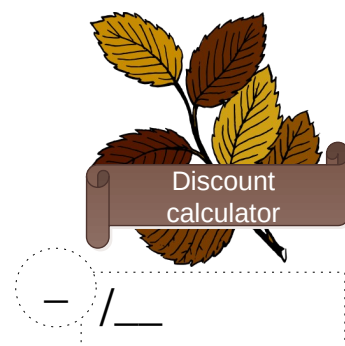
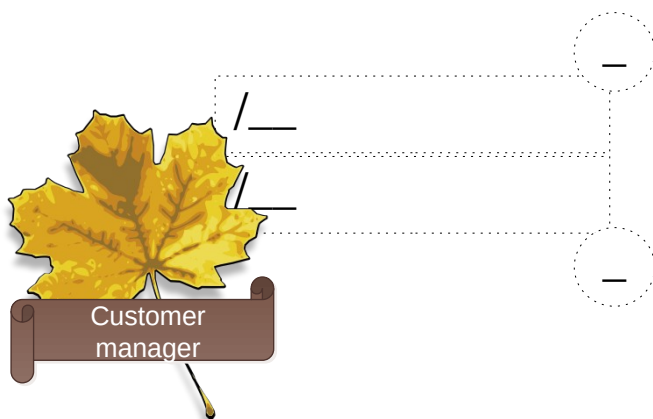
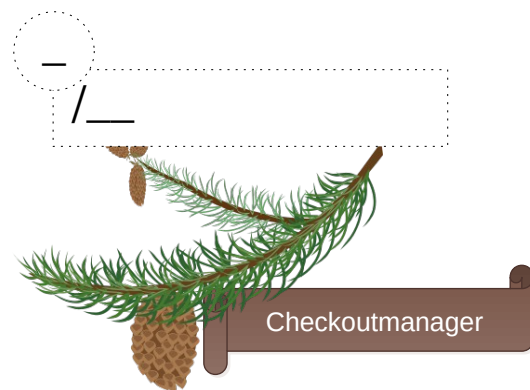
Feel free to update your Engineering Diary with your answers.





Help Skip View Their Shopping Cart

1. Draw lines to represent the calls made from Webshop::viewCart(). Follow the calls through all the services.
2. Fill in the order of the calls (the missing numbers)
3. Write the names of the remaining REST API calls



Let's do some Implementation



View the Transaction Log

Owlivia has heard that there is a database `shopdb:transactionlogdb` and she *loves* logs! Help her create a webpage to view the transaction log.

Before you begin: Make a Plan

We can access the transaction log in two ways. One is to connect directly to the database from inside the Webshop app (using e.g. the python package *psycopg2*), but this would require you to write some SQL code on top of everything else. The other way is to re-use a solution which we already have and add another postgREST service and connect to this from the webshop app. This is simpler, but if you want to try the SQL route you are welcome to do so.

Preparation Tasks

1. Stop the running application by pressing *Ctrl-C* in the console.
2. Edit *compose.yaml*
 - Make sure the *webshop* service also mounts *./Containers/Webshop* to */app*
 - Create a copy of the entire specification for the *customermanager* service and name it *logview*. Change *customerdb* to *transactionlogdb*. Change *PGRST_DB_ANON_ROLE* from *"anon"* to *"logview"*
3. Commit your edited *compose.yaml*
4. Restart the application and make sure that the new *logview* service is started.

At this stage, you are ready to start implementing the new feature in Webshop. The first edit made sure that when you save any files in the Webshop codebase, the service will be restarted (This works in a python flask app but with other frameworks and languages it may be easier to use the "develop/watch" specification in the compose file).



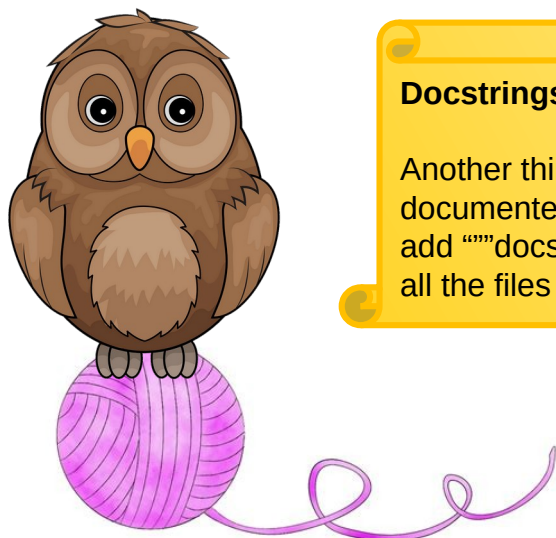
Implementation Tasks

1. Create a file *Containers/Webshop/logmanager.py* and implement a method *get_all()* that uses requests to get the url “http://logview:5000/logdata” and return the result parsed as json. For inspiration, you may look at the file *productmanager.py*.
When you save, you may notice in the console that the webshop restarts.
2. Edit *app.py* to import *logmanager*.
3. Edit *app.py* and create the new route ‘/log’.
 - You still want the shopping cart to keep track of the number of items, so you need to get the cartid and the cartitems (look at the other route functions and copy this).
 - You want to fetch the logs from *logmanager.get_all()* into a local variable.
 - There is a prepared template ‘*logs.html*’ that takes two parameters, *logs* and *cartsum*. You may look at the other route functions for the *cartsum* parameter.
4. Save (and commit!) and make sure the webshop container is restarted, then you are ready to show Owlivia the new url <http://localhost:8080/log> in your web browser. Do some shopping first to make sure that there are some log entries to view.

Update your Engineering Diary

- What have you learned?
- What works well?
- What is difficult?
- What do you need to learn more about?
- If something goes wrong when running the application, how can you find out more? How can you debug the application?

More Implementation



Docstrings

Another thing Owlivia loves is well documented code. So go ahead and add `"""docstrings"""` to all functions in all the files in the Webshop app.

Tasks

1. Add `"""docstrings"""` to all functions in all the python files.
2. Commit your changes.
3. Update your Engineering Diary
 - What have you learned?
 - Which new commands have you used?
 - What works well?
 - What is difficult?
 - What do you need to learn more about?

Show the TA

- 1) Your commit log
- 2) The Dockerfiles for PackingSystem and for Webshop
- 3) The changes you have made to compose.yaml
- 4) Your added/edited code in Webshop, and your well documented code
- 5) You starting the application
- 6) A list of the running containers
- 7) The web page <http://localhost:8080/log>
- 8) The web page <http://localhost:8080/health>
- 9) The crossword that helped put Barry the Bear to sleep
- 10) The dependency map that you helped Briant the Ant create
- 11) The two call graphs you drew together with Skip the Squirrel



Summary

My name is _____ and I know how to:

- | | |
|--------------------------|--|
| ☐ | Clone a git repository |
| ☐ | Make local commits with meaningful commit messages |
| ☐ | View a log of previous commits |
| ☐ | Write a Dockerfile |
| ☐ | Build a docker image |
| ☐ | Start a docker container |
| ☐ | Write and edit a Docker Compose file |
| ☐ | Specify how services in a docker compose file are started in a certain order |
| ☐ | Specify environment variables and volume mounts for a service |
| ☐ | Build an application with many services |
| ☐ | Start a docker compose application |
| ☐ | Write python code that fetch something from another service |
| ☐ | Write python code to create a REST API |
| ☐ | Write python code for a route function |
| ☐ | Read code in different programming languages |
| ☐ | Read text output in a console for debugging purposes |
| ☐ | Document what I do and commands I use in an Engineering Diary |
| ☐ | Reflect upon what I know and what I need to learn |



There you go. Have a flower: